# Strutture Iterative

Luca Didaci

# Table of contents


**1 Struttura iterativa `WHILE` 1**

1.1 Struttura iterativa `WHILE` . . . 1
1.2 `WHILE` - sintassi semplificata . . . 3
1.3 While - Numero di iterazioni predefinite . . . 4
1.4 Esempio . . . 5
1.5 Esercizi . . . 5
1.6 While - Iterare finché necessario . . . 7
1.7 Gestire il ciclo con break e continue . . . 8
1.8 Uscire dal ciclo con `break` . . . 9
1.9 Saltare un'iterazione con `continue` . . . 9
1.10 `WHILE` - sintassi completa . . . 9
1.11 Esercizi . . . 10

**2 ESERCIZI SUL WHILE 13**

2.1 ESERCIZIO . . . 13
2.2 ESERCIZIO . . . 13
2.3 ESERCIZIO . . . 14
2.4 ESERCIZIO . . . 14
2.5 ESERCIZIO . . . 16
2.6 ESERCIZIO . . . 16
2.7 ESERCIZIO . . . 17
2.8 ESERCIZIO . . . 18
2.9 ESERCIZIO . . . 18

**3 Struttura iterativa `FOR` 21**

3.1 Struttura iterativa `FOR` . . . 21
3.2 FOR . . . 21
3.3 Gestire il ciclo con break e continue . . . 22

**4 ESERCIZI SUL FOR 23**

4.1 ESERCIZIO . . . 23
4.2 ESERCIZIO . . . 23
4.3 ESERCIZIO . . . 24
4.4 ESERCIZIO . . . 24
4.5 ESERCIZIO . . . 25
4.6 ESERCIZIO . . . 25

4.7 ESERCIZIO . . . . . 25
4.8 ESERCIZIO . . . . . 26
4.9 ESERCIZIO . . . . . 26
4.10 ESERCIZIO . . . . . 26
4.11 ESERCIZIO . . . . . 27
4.12 ESERCIZIO . . . . . 27
4.13 ITERARE SUI DIZIONARI - ESERCIZI . . . . . 28

# 1 Struttura iterativa WHILE

## 1.1 Struttura iterativa WHILE

Spesso abbiamo necessità di ripetere un gruppo di istruzioni varie volte

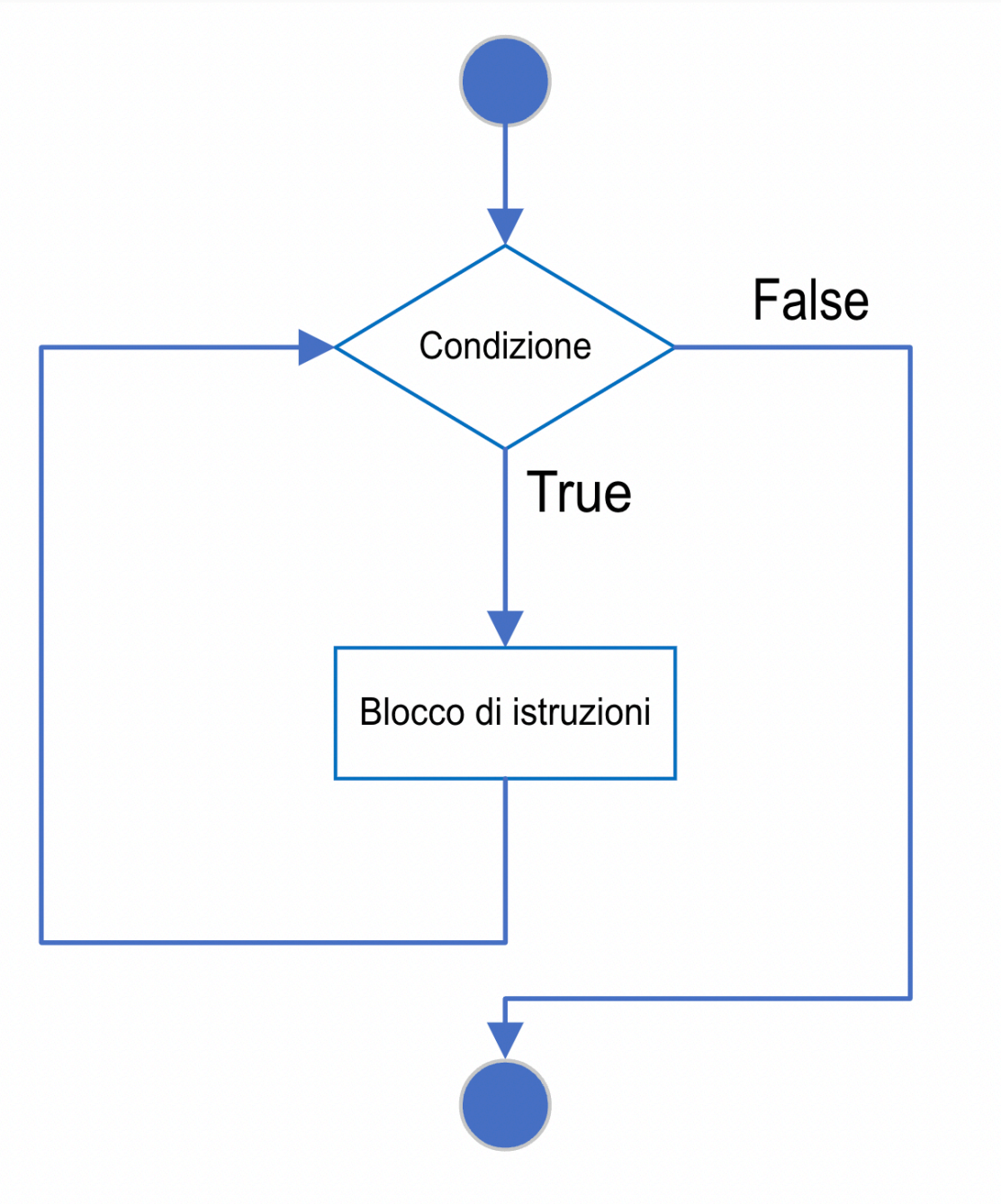
```
# TABELLINE - stampa la tabellina del 3

n_tabellina=3
i = 1
risultato= n_tabellina * i
print( n_tabellina, '*', i, '=', risultato)
i = 2
print( n_tabellina, '*', i, '=', risultato)
i = 3
print( n_tabellina, '*', i, '=', risultato)
i = 4
print( n_tabellina, '*', i, '=', risultato)
```

Se voglio stampare i primi 1000 valori questa soluzione è impraticabile...

---

Il costrutto WHILE risolve il problema




- La `condizione` viene valutata
- Se la `condizione` è vera, viene eseguito il corpo del **while**, e la `condizione` viene rivalutata
- Se la `condizione` è falsa, si esce dal ciclo

## 1.2 WHILE - sintassi semplificata

```
while (condizione): # finchè la condizione è vera...
    # corpo del while:
    # sequenza di istruzioni che vengono ripetute
    # finchè condizione è vera
# NEXT INSTRUCTION
```

- La `condizione` viene valutata
- Se la `condizione` è vera, viene eseguito il corpo del while, e la `condizione` viene rivalutata
- Se la `condizione` è falsa, si esce dal ciclo (il controllo passa a `# NEXT INSTRUCTION`)

NOTA:

Se la condizione è inizialmente `False` il corpo del `while` non verrà mai eseguito.

Se la condizione è inizialmente `True` ed il corpo del `while` non la altera, si entra in un loop infinito. **È fondamentale dunque che il corpo del `while` abbia la possibilità di alterare la condizione**.

```
while (True): # finchè la condizione è vera...
    print('fermatemiiii!')
```

---

Soluzione esercizio Tabellina

```
# i = 1
# finchè i è minore di 10 esegui le seguenti istruzioni:
#   calcola 3*i
#   stampa il risultato
#   incrementa i

i = 1
n_tabellina = 3
while ( i<=10 ):
    risultato =  n_tabellina *i
    print( n_tabellina, '*', i, '=', risultato)
    i = i+1
```

---

## 1.3 While - Numero di iterazioni predefinite

Possiamo utilizzare il costrutto `while` per eseguire un numero prefissato di iterazioni. Utilizziamo un *contatore* che viene incrementato ad ogni iterazione ed utilizzato per valutare la condizione.

Esempio:

L'utente inserisce un numero che definisce il numero di iterazioni.

In seguito il programma chiede al'utente dei valori e li stampa

```
print('quanti numeri vuoi inserire?')
n = int(input())
i=0 # contatore delle iterazioni

while (i<n):
    i = i + 1 # i viene incrementata
    # i += 1 # sintassi abbreviata
    valore = float(input())
    print('hai inserito il numero ', valore)
```

In questo esempio un contatore viene inizializzato a 0 e viene incrementato ad ogni esecuzione del corpo del `while`. La condizione `i<n` viene valutata all'inizio del ciclo (quando `i==0`) e all'inizio di ogni iterazione.

Supponiamo che `n==3`

Alla prima iterazione il valore di `i` passa da 0 a 1. Alla terza iterazione il valore passa da 2 a 3. Al termine della terza iterazione la condizione `i<n` è `False` ed il ciclo si interrompe. Come previsto vengono eseguite `n==3` iterazioni complete.

---

Errori comuni

```
n = int(input()) # dimenticare int()
i=0 # dimenticare di inizializzare

while (i<n):
    # sbagliare la condizione: i<=n, i>n, i != n

    i += 1 # dimenticare di incrementare
    # ...
```

---

## 1.4 Esempio

Scrivere un programma che chiede all'utente quanti valori desidera inserire. Il programma acquisisce i valori e stampa il massimo

```
print('quanti numeri vuoi inserire?')
n = int(input())
i=0 # contatore delle iterazioni
massimo=0
while (i<n):
    i+=1
    valore = float(input())
    if valore > massimo:
        massimo = valore

print('il massimo è ', massimo)
```

- ipotizzo che il massimo sia 0
- se il nuovo valore è maggiore del massimo, aggiorno il massimo

Questo codice assume che tutti i numeri inseriti siamo positivi. Se i numeri sono tutti negativi non funziona. Perché? Come risolvere?

## 1.5 Esercizi

Scrivere un programma che chiede all'utente un valore `n` e stampa i numeri da 1 a n

```
n = int(input())
i = 0
while (i<n):
    i = i+1
    print (i)
print ('sono uscito dal ciclo')
```

---

- Scrivere un programma che chiede all'utente un valore `n` e stampa i numeri PARI fino ad `n`. Se `n==10` il programma deve stampare `2, 4, 6, 8, 10`
- Scrivere un programma che chiede all'utente quanti valori desidera inserire. Il programma acquisisce i valori e stampa la somma

---

- Scrivere un programma che chiede all'utente quanti valori desidera inserire. Il programma acquisisce i valori e stampa la media

```
print('Quanti numeri vuoi inserire?')
n = int(input())
i = 0
s = 0
print('Inserisci i dati!')
while i<n:
    v = float(input())
    s = s+v
    i += 1

media= s/n
print('media: ', media)
```

---

- Scrivere un programma che chiede all'utente quanti valori desidera inserire. Il programma acquisisce i valori e stampa il minimo e il massimo. Il programma deve funzionare con numeri qualunque.
- Scrivere un programma che chiede all'utente quanti valori desidera inserire. Il programma acquisisce i valori e stampa la media 'robusta' calcolata eliminando il minimo ed il massimo.
- Scrivere un programma che chiede all'utente quante frasi desidera inserire. Il programma acquisisce le frasi e stampa la frase più lunga. Hint: usare `len(stringa)`

---

- Conversione base 2-> base 10. Scrivere un programma che chieda all'utente il numero di bit che intende utilizzare, e poi chieda il valore dei bit a partire dal *msb* (most significant bit, il bit più significativo). Il programma deve stampare il valore in base 10.

```
# SOLUZIONE:
print("Preparati ad inserire un numero intero espresso in base 2, a partire da MSB")
print("Quanti bit?")

n= int(input())
print("Ok, mi preparo a converitire un numero di ", n, " bit. Comincia l'inserimento")
i=0
v=0
while i<n:
    v= v*2
    b= int(input())
```

```
    v= v+b
    i+=1
print('In base 10: ---> ',v)
```

---

## 1.6 While - Iterare finché necessario

Possiamo utilizzare il costrutto `while` per eseguire delle iterazioni finchè non si verifica un particolare evento. Il numero di iterazioni **non** è prefissato in anticipo, ma può dipendere da una condizione che viene valutata ad ogni iterazione.

```
while condizione:
    # Blocco di istruzioni ripetute finché la condizione è vera
    # Eventuale modifica della condizione
```

Il ciclo continuerà a eseguire il blocco di codice finché la condizione rimane `True`. Non appena la condizione diventa `False`, l'esecuzione del ciclo si interrompe.

---

**Esempio: inserisci numeri; 0 per terminare**

```
valore = int(input())
while(valore != 0):
    print('hai inserito', valore)
    valore = int(input())
```

Nota:

- Il primo `valore` viene inserito prima del ciclo, e viene utilizzato per valutare la *condizione* la prima volta
- se il primo `valore` è zero nessuna iterazione viene eseguita
- quando `valore==0` la condizione è `False` ed il ciclo termina. `print('hai inserito', valore)` non viene eseguita per il valore 0.

---

**Esempio: inserisci numeri; 0 per terminare**

È possibile modificare il ciclo in modo che anche il primo input sia all'interno del ciclo. Dato che `valore` viene utilizzato nella condizione, occorre assegnare un valore fittizio per forzare l'ingresso nel ciclo.

```
valore = None
while(valore != 0):
    valore = int(input())
    print('hai inserito', valore)
```

In questo caso l'istruzione

`print('hai inserito', valore)`

viene eseguita anche nell'ultima iterazione, quando `valore==0`.

---

**Variabile sentinella**

Possiamo utilizzare una variabile sentinella per gestire la condizione.

```
continua = True
while(continua):
    valore = int(input())
    print('hai inserito', valore)
    continua = (valore!= 0)
```

Se `valore!= 0` il valore `True` viene assegnato a `continua`, ed il ciclo continua. Altrimenti `continua == False` ed il ciclo si interrompe.

Nota: `continua = (valore!= 0)` equivale a

```
if valore == 0:
    continua= False
else:
    continua = True
```

## 1.7 Gestire il ciclo con break e continue

Nel ciclo `while` possiamo usare due parole chiave speciali per controllare l'esecuzione.

- `break`: interrompe completamente il ciclo e passa all'istruzione successiva fuori dal `while`. Si noti che in caso di cicli annidati (*nested*), cioè inseriti uno dentro l'altro, il `break` interrompe solo il ciclo più interno.
- `continue`: salta l'iterazione corrente e passa all'iterazione successiva, senza eseguire il resto del codice all'interno del ciclo ma senza interrompere il ciclo

## 1.8 Uscire dal ciclo con `break`

**Esempio**: chiedere all'utente di inserire un numero finché non digita 0:

```
while True:  # Un ciclo teoricamente infinito
    print("Inserisci un numero (0 per uscire): ")
    numero = int(input())
    if numero == 0:
        print("Hai scelto di terminare il programma.")
        break  # Esce dal ciclo
    print("Hai inserito:", numero)
```

## 1.9 Saltare un'iterazione con `continue`

La parola chiave continue permette di saltare l'iterazione corrente e passare alla successiva.

**Esempio**: stampare i numeri da 1 a 10, ma saltare i multipli di 3

```
numero = 0

while numero < 10:
    numero += 1
    if numero % 3 == 0:
        # se il resto della divisione tra numero e 3 è ==0
        continue  # Salta il resto del codice e passa alla prossima iterazione
    print(numero)
```

## 1.10 `WHILE` - sintassi completa

https://docs.python.org/it/3.13/reference/compound_stmts.html#while

```
while condizione:
    # Blocco di codice eseguito finché la condizione è vera
else:
    # Blocco eseguito solo se il ciclo termina normalmente (senza break)
```

- Il blocco dentro `while` viene eseguito finché `condizione == True`.
- Se il ciclo termina normalmente (perché `condizione == False`), viene eseguito il blocco `else`.
- Se il ciclo viene interrotto con `break`, il blocco `else` non viene eseguito.

## 1.11 Esercizi

- Scrivere un programma che contiene un numero segreto e chiede all'utente di indovinarlo. Sono ammessi 5 tentativi. La condizione di uscita deve quindi tener conto sia del valore inserito dall'utente sia del numero di tentativi.
- Scrivere un programma che chiede all'utente dei numeri e li somma. Il programma termina quando l'utente inserisce 0. Formulate una soluzione con il primo input prima del ciclo e un'alra soluzione cn tutti gli input all'interno del ciclo.

---

- Scrivere un programma che chiede all'utente dei numeri e li somma. Il programma termina quando la somma è maggiore di 100. Se dopo 10 iterazioni la somma non è ancora maggiore di 100, il programma termina lo stesso.

```
print('Inserisci i dati, termino quando la somma eccede 100 o dopo 10 inserimenti!')
s=0
i=0
while s<=100 and i<10:
    # DE MORGAN
    # not(s>100 or i>=10)
    v = int(input())
    s = s+v
    i += 1
print('somma:', s)
print('numero inserimenti:', i)

# VARIANTE:
# stampare un messaggio che spieghi se il ciclo è terminato
# perché somma >100 o perchè si sono superate le 10 iterazioni
```

---

- Scrivere un programma che chiede all'utente un insieme di numeri POSITIVI o NEGATIVI (ma non nulli) e ne calcola la somma. L'utente inserisce zero per uscire.
- Scrivere un programma che chiede all'utente un insieme di numeri qualunque e ne calcola la somma. L'utente inserisce il carattere `q` per uscire. Suggerimento: si supponga che l'utente non inserisca mai input errati. Se il valore inserito non è `q`, allora sicuramente è convertibile in numero.
- Scrivere un programma che chiede all'utente un insieme di numeri POSITIVI e ne calcola la media. L'utente inserisce zero per uscire. Variante: media robusta (eliminando il massimo e minimo)

---

- Dato un numero intero positivo, contare il numero di cifre. NB è richiesto di manipolare il numero, non la sua rappresentazione mediante stringa!

```
# Soluzione
print("Inserisci un numero intero, calcolerò il numero di cifre")
n= int(input())
i=0
while n>=1:
    i= i+1
    n = n/10
print('Numero di cifre ---> ',i)
```

---

- Dato un numero intero positivo, stampare il numero al contrario. NB è richiesto di manipolare il numero, non la sua rappresentazione mediante stringa!

```
# Soluzione
n_reverse=0
print('Inserisci un numero')
n= int(input())
while n>=1:
    resto= n%10
    n= int(n/10)
    n_reverse = n_reverse*10
    n_reverse = n_reverse + resto
print(n_reverse)
```

---

- Dati due numeri interi positivi, trovare il loro massimo comune divisore (MCD) usando l'Algoritmo di Euclide https://it.wikipedia.org/wiki/Algoritmo_di_Euclide

---

- Scrivere un programma che dica se $p$ è un numero è primo. Varianti: se $p$ è un numero primo, dire se è un numero primo di Sophie Germain[1]. Dire se è un numero primo di Mersenne[2]

---

[1] https://it.wikipedia.org/wiki/Numero_primo_di_Sophie_Germain
[2] https://it.wikipedia.org/wiki/Numero_primo_di_Mersenne

- Scrivere un programma che contiene un numero segreto e chiede all'utente di indovinarlo. Variante: Il computer sceglie un numero segreto tra 1 e 100, e l'utente ha solo 5 tentativi per indovinarlo. Dopo ogni tentativo il programma deve dire "Troppo alto" o "Troppo basso" finché l'utente indovina o esaurisce i tentativi. Suggerimento: usa la funzione `random.randint()` per generare un numero casuale.

```
import random

# genera un numero (pseudo) casuale tra 1 e 100
numero_segreto = random.randint(1,100)

n= 5
i = 0
while i<n:
    i = i+1
    print('indovina il numero')
    v = int(input())
    if v == numero_segreto:
        print('INDOVINATO!')
        break
    elif v<numero_segreto: # riferito a IF
        print('Hai inserito un numero troppo piccolo')
    else:
        print('Hai inserito un numero troppo grande')
else: # riferito a WHILE
    print('mi dispiace, hai esautito i tentativi')
    print('il numero era ', numero_segreto)


# La maggior parte dei linguaggi di programmazione hanno la struttura WHILE
# senza la clausola ELSE
# Formulate una soluzione alternativa SENZA la clausola ELSE del WHILE
# in modo che si possa implementare in altri linhguaggi di programmazione
```

# 2 ESERCIZI SUL WHILE

## 2.1 ESERCIZIO

Chiedi all'utente di inserire numeri finché digita 0. Salva i numeri in una lista e stampala alla fine.

```
numeri = []
print("Inserisci un numero (0 per terminare): ")
numero = int(input())
#numero = int(input("Inserisci un numero (0 per terminare): "))

while numero != 0:
    numeri.append(numero)
    numero = int(input("Inserisci un numero (0 per terminare): "))

print("Hai inserito:", numeri)
```

## 2.2 ESERCIZIO

Somma dei numeri pari

Obiettivo: Inserisci numeri finché l'utente scrive -1. Salva i numeri pari in una lista e calcola la loro somma.

```
pari = []
numero = int(input("Inserisci un numero (-1 per terminare): "))

while numero != -1:
    if numero % 2 == 0:
        pari.append(numero)
    numero = int(input("Inserisci un numero (-1 per terminare): "))

print("Numeri pari:", pari)

somma=0
i=0
while i<len(pari):
```

```
    somma += pari[i]
    i += 1


print("Somma dei pari:", somma))
```

## 2.3 ESERCIZIO

Lista al contrario

Chiedi n numeri all'utente, poi stampa la lista in ordine inverso usando while.

```
numeri = []
i = 0

n=5
while i < n:
    num = int(input(f"Inserisci il numero {i+1}: "))
    numeri.append(num)
    i += 1

# Stampa al contrario con while
i = len(numeri) - 1
print("Numeri al contrario:")
while i >= 0:
    print(numeri[i])
    i -= 1

# soluzione alternativa
i =  - 1
print("Numeri al contrario:")
while i >= -n:
    print(numeri[i])
    i -= 1
```

## 2.4 ESERCIZIO

Rimuovi tutti i negativi da una lista

Inserisci numeri finché digiti `q`, poi rimuovi tutti i numeri negativi dalla lista.

```
numeri = []
elemento = input("Inserisci un numero (q per terminare): ")

while elemento != 'q':

    numeri.append(int(elemento))
    elemento = input("Inserisci un numero (q per terminare): ")

# Rimozione negativi
# ATTENZIONE: incremento solo se non negativo!
i = 0
while i < len(numeri):
    if numeri[i] < 0:
        numeri.pop(i)
    else:
        i += 1

print("Lista senza numeri negativi:", numeri)
```

questo non funziona; perché?

```
i = 0
while i < len(numeri):
    if numeri[i] < 0:
        numeri.pop(i)

    i += 1
```

questo non funziona; perché?

```
i = 0
while i < len(numeri):
    idx= len(numeri)- i-1
    el = numeri[idx]
    if el < 0:
        #numeri.pop(idx)
        del numeri[idx]

    i += 1
print(numeri)
```

questo non funziona; perché?

```
i = -1
while i >= -len(numeri):
    el = numeri[i]
    print(el)
    if el < 0:
        #numeri.pop(idx)
        del numeri[i]


    i -= 1
print(numeri)
```

## 2.5 ESERCIZIO

Conta occorrenze di una parola

Data una lista di stringhe, chiedi una parola da cercare e stampa quante volte è presente in lista

```
parole = ['cane', 'gatto', 'gatto', 'topo', 'gatto']


cerca = input("Quale parola vuoi contare? ")
conteggio = 0
i = 0

while i < len(parole):
    if parole[i] == cerca:
        conteggio += 1
    i += 1

print(f"La parola '{cerca}' è stata inserita {conteggio} volte.")
```

## 2.6 ESERCIZIO

Elimina i duplicati

Inserisci numeri finché l'utente digita -1. Crea una nuova lista senza duplicati.

```
numeri = []
numero = int(input("Inserisci un numero (-1 per terminare): "))

while numero != -1:
    numeri.append(numero)
```

```
    numero = int(input("Inserisci un numero (-1 per terminare): "))

unici = []
i = 0
while i < len(numeri):
    if numeri[i] not in unici:
        unici.append(numeri[i])
    i += 1

print("Lista senza duplicati:", unici)
```

oppure, senza usare `not in`:

(provare con while-else o con variabile sentinella)

```
numeri = []
numero = int(input("Inserisci un numero (-1 per terminare): "))

while numero != -1:
    numeri.append(numero)
    numero = int(input("Inserisci un numero (-1 per terminare): "))

print('!')
unici = [numeri[0]]
i = 1
while i < len(numeri):
    j=0
    while j < len(unici):
        if numeri[i] == unici[j]:
            break
        j+=1
    else:
        unici.append(numeri[i])
    i += 1

print("Lista senza duplicati:", unici)
```

## 2.7 ESERCIZIO

Interrompi se inserisci 3 volte lo stesso numero. Il numero inserito deve essere compreso tra 0 e 99

(senza usare dizionari)

```
n = 100
numero = int(input("Inserisci un numero (-1 per terminare): "))
numeri=[0]*n # crea una lista di n elementi, tutti a 0
while numero !=-1:
    numeri[numero]+=1
    if numeri[numero] ==3:
        break
    numero = int(input("Inserisci un numero (-1 per terminare): "))
```

## 2.8 ESERCIZIO

Chiedi 10 numeri. Calcola la media dei numeri in posizione pari (indice 0, 2, 4…).

```
numeri = []
i = 0

while i < 10:
    num = int(input(f"Inserisci il numero {i+1}: "))
    numeri.append(num)
    i += 1

somma = 0
conta = 0
i = 0

while i < len(numeri):
    if i % 2 == 0:
        somma += numeri[i]
        conta += 1
    i += 1

media = somma / conta
print("Media dei numeri in posizione pari:", media)
```

## 2.9 ESERCIZIO

Palindroma o no?

Verifica se una lista forma una parola palindroma (uguale da sinistra e da destra).

```
frase = ['a', 'b', 'b', 'a']
```

```
# frase = ['a','l', 'b', 'a']

i = 0 #indice che parte da sinistra
j = len(frase) - 1 #indice che parte da destra
palindroma = True

while i < j:
    if frase[i] != frase[j]:
        palindroma = False
        break
    i += 1
    j -= 1


if palindroma:
    print("La parola è palindroma!")
else:
    print("La parola NON è palindroma.")
```

# 3 Struttura iterativa FOR

## 3.1 Struttura iterativa FOR

Questa parte è da svolgere DOPO aver visto il materiale sulle LISTE

Si riportano qui gli elementi essenziali della funzione `range()`

La funzione `range()` restituisce un oggetto iterabile che produce una sequenza di numeri interi. La sintassi è `range(start, stop, step)`

- `start`: valore iniziale della sequenza (incluso). Default: 0
- `stop`: valore finale (escluso)
- `step`: incremento tra un elemento e il successivo. Default: 1

L'oggetto `range` non è una lista, ma immagazzina i parametri essenziali per la creazione di una lista di intei (il primo elemento, lo step, l'ultimo elemento). L'oggetto `range` può essere convertito in una lista tramite il comando `list()`[1].

https://docs.python.org/2/library/functions.html#range

## 3.2 FOR

In Python, il ciclo `for` è uno **statement di controllo del flusso** che consente di iterare su una sequenza (iterable), come una *lista*, una *tupla*, un *dizionario*, una *stringa* o un oggetto che implementa il protocollo dell'iteratore.

Il ciclo `for` in Python è semanticamente diverso da quello in altri linguaggi come C o Java: non esegue iterazioni basate su un'espressione booleana (come fa invece il `while`), ma scorre direttamente gli elementi di un oggetto iterabile.

**SINTASSI COMPLETA**

```
for <variabile> in <iterabile>:
    <blocco_di_codice>
else:
    <blocco_opzionale_else>
```

[1] tecnicamente `list` è una classe built-in

La keyword `else` definisce un blocco di codice opzionale che viene eseguito una sola volta alla fine del ciclo, *solo se il ciclo non è stato interrotto da un break.*

## 3.3 Gestire il ciclo con break e continue

Nel ciclo `for`, esattamente come in `while`, possiamo usare due parole chiave speciali per controllare l'esecuzione.

- `break`: interrompe completamente il ciclo e passa all'istruzione successiva fuori dal `for`. Si noti che in caso di cicli annidati (*nested*), cioè inseriti uno dentro l'altro, il `break` interrompe solo il ciclo più interno.
- `continue`: salta l'iterazione corrente e passa all'iterazione successiva, senza eseguire il resto del codice all'interno del ciclo ma senza interrompere il ciclo

# 4 ESERCIZI SUL FOR

Questa parte è da svolgere DOPO aver visto il materiale sulle LISTE

Alcuni esercizi sono gli stessi proposti per il while (o molto simili). Notate come la soluzione di alcuni esercizi è più semplice se uso il `FOR` invece del `WHILE`

## 4.1 ESERCIZIO

Stampa la lista in ordine inverso usando FOR

```
numeri = [1, 2,3,4,5]
n=len(numeri)
for i in range(n-1,-1,-1):
    print(numeri[i])
```

## 4.2 ESERCIZIO

Conta occorrenze di una parola

Data una lista di stringhe, chiedi una parola da cercare e stampa quante volte è presente in lista

```
parole = ['cane', 'gatto', 'gatto', 'topo', 'gatto']

cerca = input("Quale parola vuoi contare? ")
conteggio = 0

for el in parole:
    if el == cerca:
        conteggio += 1

print(f"La parola '{cerca}' è stata inserita {conteggio} volte.")
```

## 4.3 ESERCIZIO

Crea una nuova lista senza duplicati.

```
numeri = [1,2,3,2,3,4,3,4,5,6,7]

unici = []

for el in numeri:
    if el not in unici:
        unici.append(el)

print("Lista senza duplicati:", unici)
```

oppure, senza usare `not in`:

```
numeri = [1,2,3,2,3,4,3,4,5,6,7]

unici = []

for el_1 in numeri:
    for el_2 in unici:
        if el_1 == el_2:
            break
    else:
        unici.append(el_1)

print("Lista senza duplicati:", unici)
```

## 4.4 ESERCIZIO

Calcola la media dei numeri in posizione pari (indice 0, 2, 4...).

```
numeri = [1, 4, 3 , 8, 5, 3, 7, 1, 9] # mean == 5
somma=0
conta=0
for i in range(0,len(numeri),2):
    somma += numeri[i]
    conta+=1
media = somma/conta

print("Media dei numeri in posizione pari:", media)
```

## 4.5 ESERCIZIO

Palindroma o no?

Verifica se una lista forma una parola palindroma (uguale da sinistra e da destra).

```
frase = ['a', 'b', 'b', 'a']
# frase = ['a','l', 'b', 'a']

for i in range(0, int(len(frase)/2)):
    j = len(frase)-i-1
    if frase[i] != frase[j]:
        print("La parola NON è palindroma.")
        break
else:
    print("La parola è palindroma!")
```

## 4.6 ESERCIZIO

Data una lista, conta il numero di elementi pari e elementi dispari

```
lista = [ 1,2,3,4,5,6,7,8,9]

n_pari = 0
n_dispari = 0

for el in lista:
    if el%2==0:
        n_pari += 1
    else:
        n_dispari += 1

print('pari: ', n_pari, 'dispari: ', n_dispari )
```

## 4.7 ESERCIZIO

Data una lista, stampa gli elementi maggiori di 10

```
lista = [ 1,3,5,10, 21, 44]

for el in lista:
```

```
    if el>10:
        print (el)
```

## 4.8 ESERCIZIO

Data una lista, crea una nuova lista raddoppiando i valori negativi e lasciando inalterati i valori positivi

```
lista = [ 1,-3,5,10, -4, 4]
lista2=[]
for el in lista:
    if el>0:
        lista2.append(el)
    else:
        lista2.append(2*el)
print(lista2)
```

## 4.9 ESERCIZIO

Data una lista, raddoppia i valori negativi e lascia inalterati i valori positivi

```
lista = [ 1,-3,5,10, -4, 4]

for i in range(len( lista)):
    if lista[i]<0:
        lista[i]*=2
print(lista)
```

## 4.10 ESERCIZIO

Trova la posizione del primo numero negativo

```
lista = [ 1,-3,5,10, -4, 4]
# lista = [1,2,3,4,5]

trovato = False
for i in range(len( lista)):
    if lista[i]<0:
        i_neg = i
        trovato = True
        break
```

```
if trovato:
    print ('posizione del primo numero negativo: ',i_neg)
else:
    print('solo valori positivi')
```

## 4.11 ESERCIZIO

Conta le vocali in una lista di lettere

```
lettere = ['l', 'e', 'o', 'p', 'a', 'r', 'd', 'o']
vocali = ['a', 'e', 'i', 'o', 'u']
conteggio = 0

for l in lettere:
    if l in vocali:
        conteggio += 1

print("Numero di vocali:", conteggio)
```

## 4.12 ESERCIZIO

Confronta due liste e trova gli elementi comuni

Il programma deve produrre una lista di elementi comuni NON duplicati

```
lista1 = [7, 11, 13, 19]
lista2 = [2,3,5,7, 19, 23,29]
comuni = []

for el in lista1:
    if el in lista2 and el not in comuni:
        comuni.append(el  )

print("Elementi comuni:", comuni)
```

Soluzione che non usa l'operatore `in lista`. Il codice esegue cicli non necessari (analizzate il codice passo-passo). COME SI PUO' MIGLIORARE?

```

lista1 = [7, 11, 13, 19, 7]
lista2 = [2, 3, 5, 7, 19, 23, 29]
comuni = []
```

```
for el1 in lista1:
    for el2 in lista2:
        if el1 != el2:
            continue
        else: # el1==el2
            for el3 in comuni:
                if el1 == el3:  # esiste già
                    break
            else: # raggiunge il termine naturale del for senza break
                comuni.append(el1)

print("Elementi comuni:", comuni)
```

Codice migliorato:

```
lista1 = [7, 11, 13, 19, 7]
lista2 = [2, 3, 5, 7, 19, 23, 29]

comuni = []
for el1 in lista1:
    for el2 in lista2:
        if el1 == el2:
            for el3 in comuni:
                if el1 == el3:
                    break  # già presente
            else:
                comuni.append(el1)  # non è stato trovato, quindi lo aggiungiamo
            break  # usciamo da el2 - corrispondenza già trovata

print("Elementi comuni:", comuni)
```

## 4.13 ITERARE SUI DIZIONARI - ESERCIZI

*in progress*

Questa parte è da svolgere DOPO aver visto il materiale sui dizionari.

→ **Esercizio**.

Considerate due liste `l1` ed `l2` della stessa lunghezza. La lista `l1`contiene solo elementi non mutabili, unici. Scrivete un programma che, iterando su entrambe le liste, crea un dizionario usando come chiavi gli elementi della prima lista e come valore gli omologhi elementi della seconda lista

→ **Esercizio**. Dato un dizionario, creare una lista con i suoi valori. Utilizzate sia il metodo `values()` sia l'accesso ai valori tramite chavi e iterazioni.

→ **Esercizio**.

Considerate due liste `l1` ed `l2` della stessa lunghezza, contenenti rispettivamente città e temperature. Le città possono essere ripetute. Questo significa che per quella città la temperatura è stata rilevata più volte.

```
l1= ['ca', 'ge', 'ca']
l2= [30, 27, 28]
```

create un dizionario con chiave il nome della città e come valori la *lista di temperature*. Nel caso la città compaia solo una volta, il valore deve essere comunque una lista ocn un solo elemento

→ **Esercizio**.

scandite un dizionario. Il ciclo si deve interrompere quando `chiave ==0`

Varianti: - condizioni analoghe per il valore. - se una condizione diventa `True`, la chiave successiva deve essere saltata